

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

CAMPUS DE GANDIA

DOCUMENTO DE DISEÑO

Asignatura: Proyecto Aplicaciones de Biometría y Medio Ambiente

Fecha de Entrega: 20/09/2026

Pelin Kaynak

Índice

1. Introducción **3**
2. Descripción del sistema **3**
3. Diseños **4**
3.1 Arduino **4**
LED 4
Medidor 5
Publicador 5
PuntoSerie 6
EmisoraBLE 7
ServicioEnEmisora y Caracteristica 8
3.2 Android (aplicación del teléfono) **9**
MainActivity 10
Utilidades 11
TramaIBeacon 12
3.3 Lógica del teléfono **13**
LogicaFakeTelefono 13
PeticionarioREST 14
3.4 Servidor REST **15**
3.5 Lógica del negocio **16**
3.6 Base de datos **17**
3.7 Lógica del navegador **17**
3.8 Web (UX del navegador) **18**
4. Diseños que he quitado **19**

1. Introducción

En este documento muestro los diseños de todas las partes del proyecto del Sprint 0. Este documento reúne y actualiza mis dos documentos anteriores: el Documento de diseño y Diseños y prompts para generar nuevo código. Durante el sprint he cambiado el código y por eso también he cambiado los diseños, para que los diseños y el código sean iguales.

Para hacer los diseños he seguido las reglas de la asignatura (Logical Design & Reverse Engineering Specification v3):

Tipos: uso N (naturales), Z (enteros), R (reales), B o T/F (booleanos) y Text (texto). Las listas se escriben con [ ] y los grupos de datos con ( ).

Flechas a la izquierda: la flecha --> muestra los datos que entran en el método. La flecha <-- muestra los datos que salen.

Flechas a la derecha: --> significa que el método cambia el objeto, <-- significa que solo lo lee y --x significa que es estático.

Privado y público: los atributos y los métodos privados están dentro de la caja. Los métodos públicos están sobre la línea izquierda de la caja.

Callbacks: no los dibujo como parámetros. Solo escribo los datos importantes que entran y salen.

He organizado el documento por partes, igual que las carpetas del proyecto (src/): arduino, android, lógica del teléfono, servidor REST, lógica del negocio, base de datos, lógica del navegador y web.

En cada parte he añadido el diseño actualizado. También explico qué he cambiado respecto a las tareas anteriores y por qué.

2. Descripción del sistema

El sistema mide un valor de CO2 y lo muestra en una página web. Funciona así:

Placa → Bluetooth → Teléfono → Servidor REST → Base de datos → Web

1. **La placa (Arduino)** obtiene una medición de CO2. En este sprint el valor es ficticio (123). La placa envía la medición por Bluetooth en una trama iBeacon.
2. **El teléfono (Android)** escucha las tramas iBeacon. Cuando encuentra una trama de mi placa, saca los datos y los muestra en la pantalla. Después los envía al servidor.
3. **El servidor REST** recibe la medición y la pasa a la lógica del negocio.
4. **La lógica del negocio** guarda la medición en la **base de datos** con la fecha y la hora.
5. **La página web** pide la última medición al servidor y la muestra.

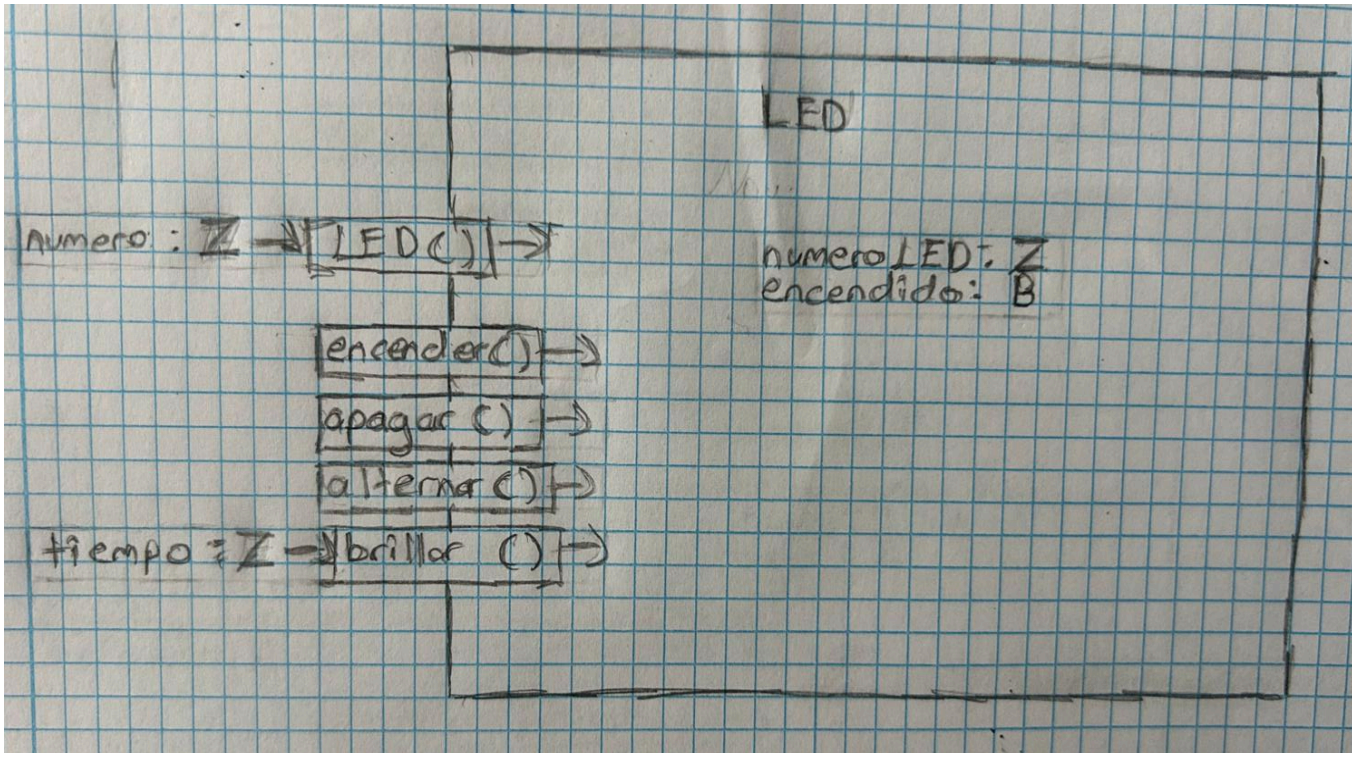
Cómo envía los datos la placa: la trama iBeacon tiene tres partes importantes:

- **UUID (16 bytes):** los 14 primeros bytes son `EPSG-GTI-PELIN`. Así el teléfono sabe que la trama es de mi placa y no de otra placa de la clase. Los 2 últimos bytes son el número del sensor (**IdSensor**). Mi placa tiene el número 1.
- **major:** el tipo de medición (11 = CO2) y un contador.
- **minor:** el valor de la medición.

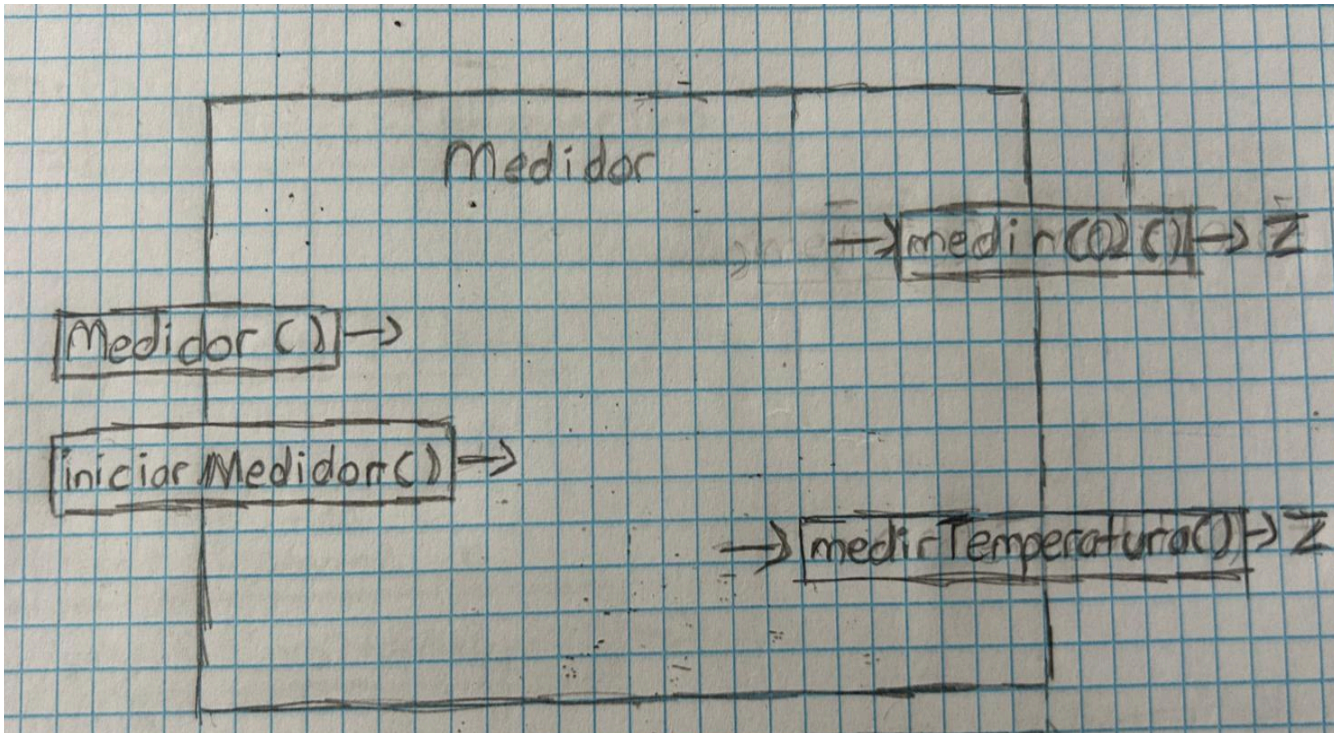
3. Diseños

3.1 Arduino

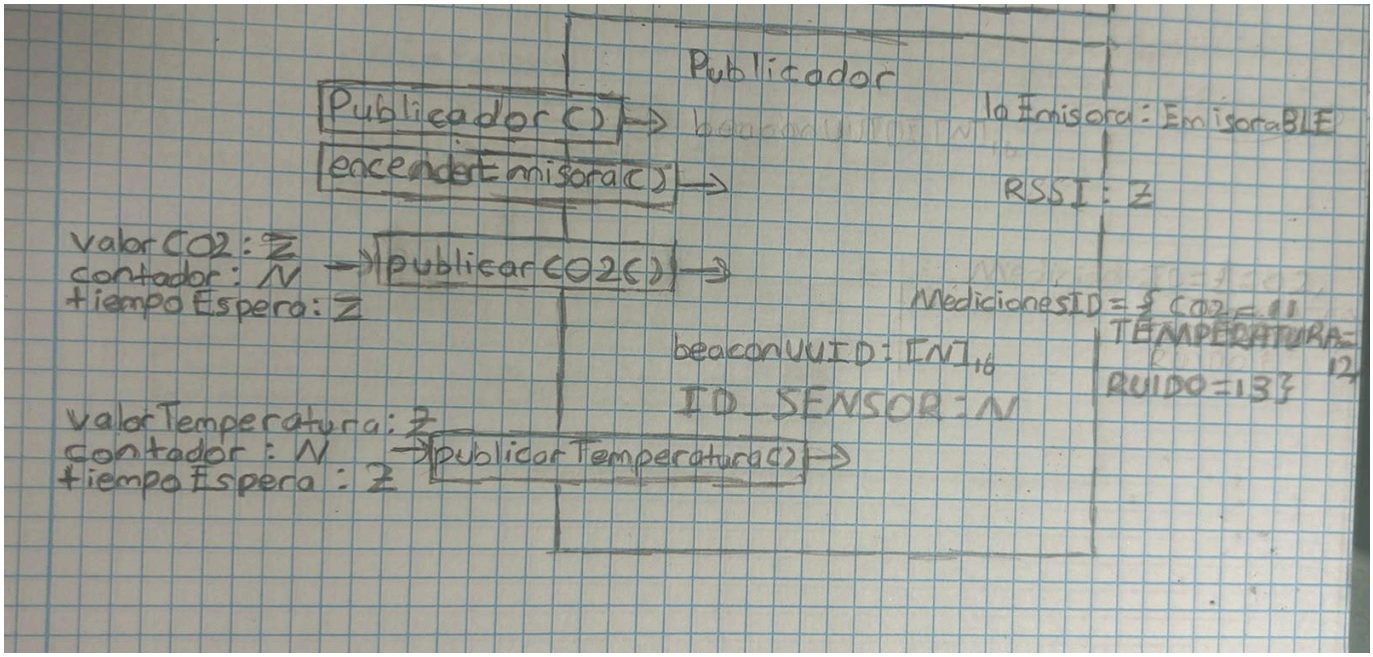
LED



Medidor



Publicador

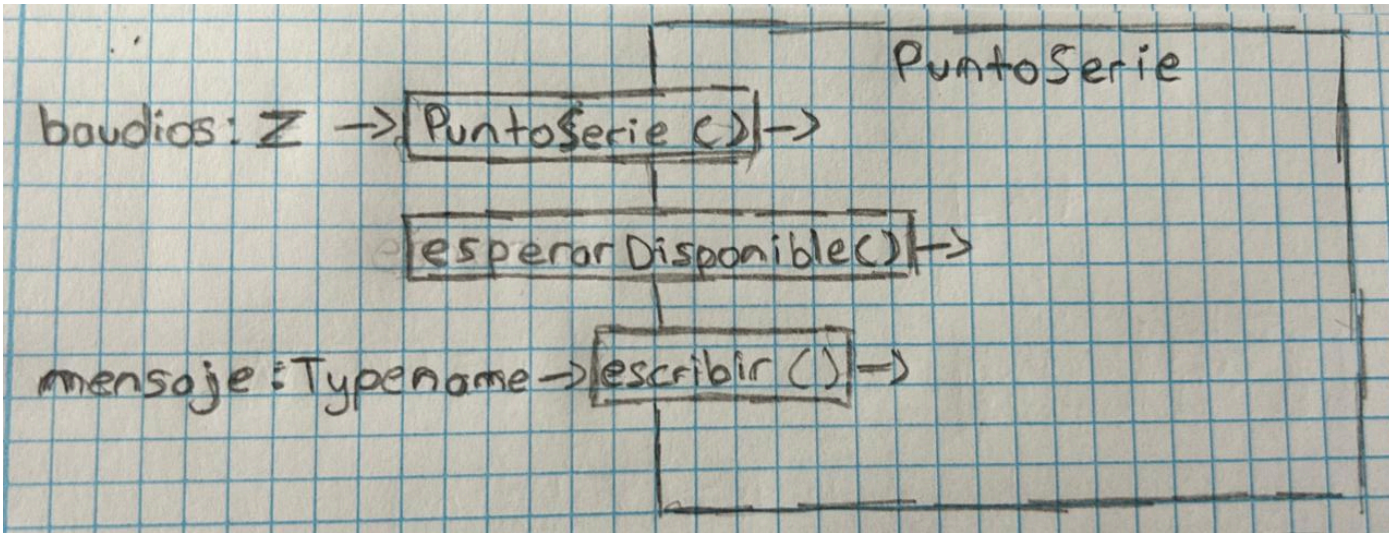


He añadido ID_SENSOR = 1, el número del sensor.He quitado la columna UUID. Ahora guardo el número del sensor (IdSensor), porque en el futuro puede haber más de un sensor.

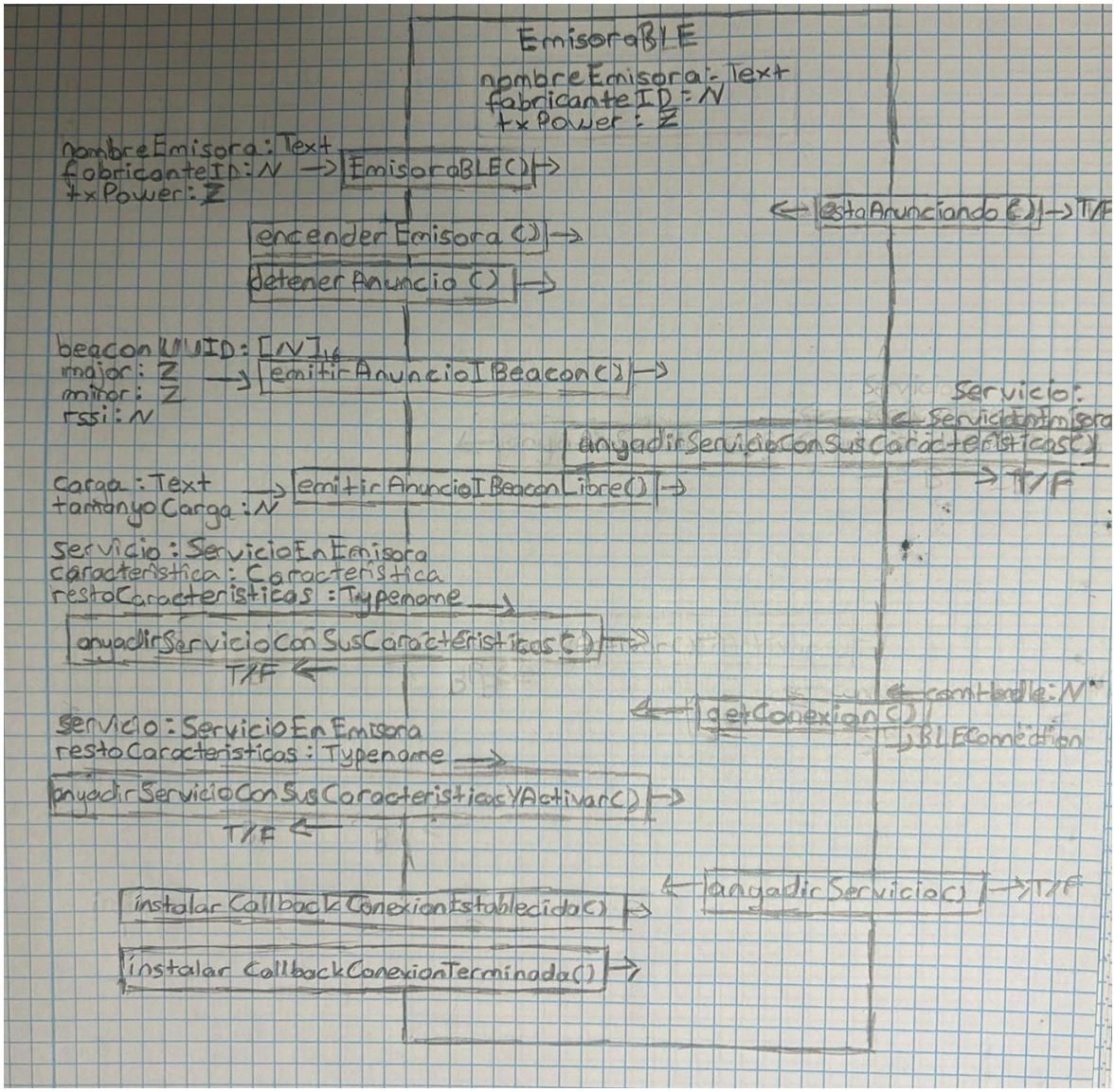
El UUID ahora es "EPSG-GTI-PELIN" + ID_SENSOR. Antes era igual para todas las placas de la clase.

He cambiado el nombre de la emisora a PelinIBeacon para encontrar mi placa fácilmente en la aplicación nRF Connect.

PuntoSerie



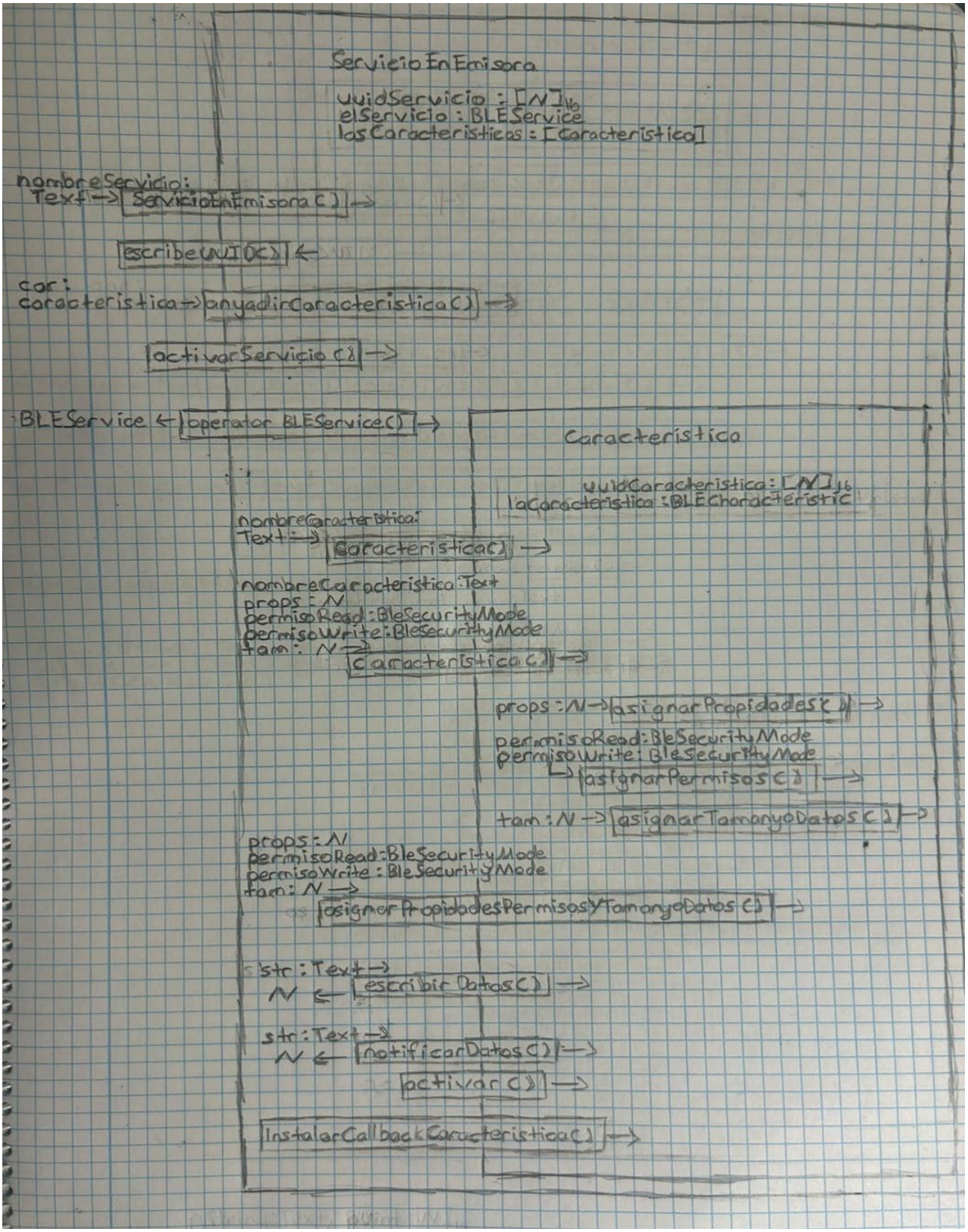
EmisoraBLE



He añadido los métodos que faltaban: anyadirServicio(), instalarCallbackConexionEstablecida() e instalarCallbackConexionTerminada().

En el código hay dos encenderEmisora(). Una de ellas recibe callbacks. Como los callbacks no se dibujan, las dos tienen el mismo diseño y la dibujo una sola vez.

ServicioEnEmisora y Caracteristica



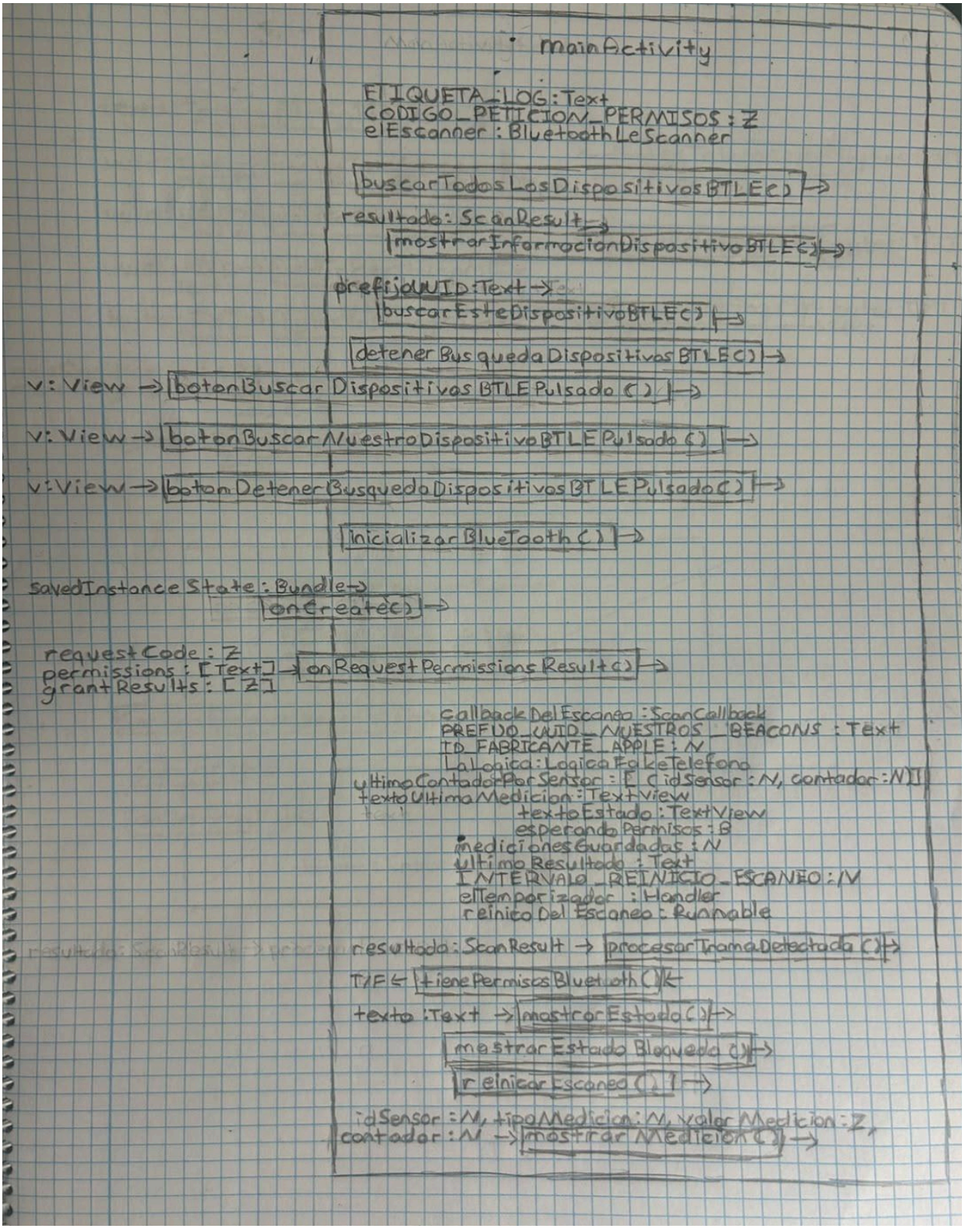
He añadido operator BLEService(). Es una conversión de tipo de C++: permite usar un ServicioEnEmisora donde Bluefruit necesita un BLEService.

He añadido instalarCallbackCaracteristicaEscrita(). Antes no lo dibujaba porque recibe un callback. Ahora lo dibujo sin el callback, como dicen las reglas.

3.2 Android (aplicación del teléfono)

Las clases de la aplicación son MainActivity, TramaIBeacon y Utilidades. .

MainActivity



buscarEsteDispositivoBTLE() ahora recibe prefijoUUID en vez de dispositivoBuscado. Antes la aplicación buscaba la placa por su nombre, pero en algunos teléfonos no funcionaba. Ahora busca las tramas cuyo UUID empieza por EPSG-GTI-PELIN.

Nuevo método procesarTramaDetectada(): saca de la trama el IdSensor, el tipo, el contador y el valor, y envía la medición con la lógica del teléfono (laLogica).

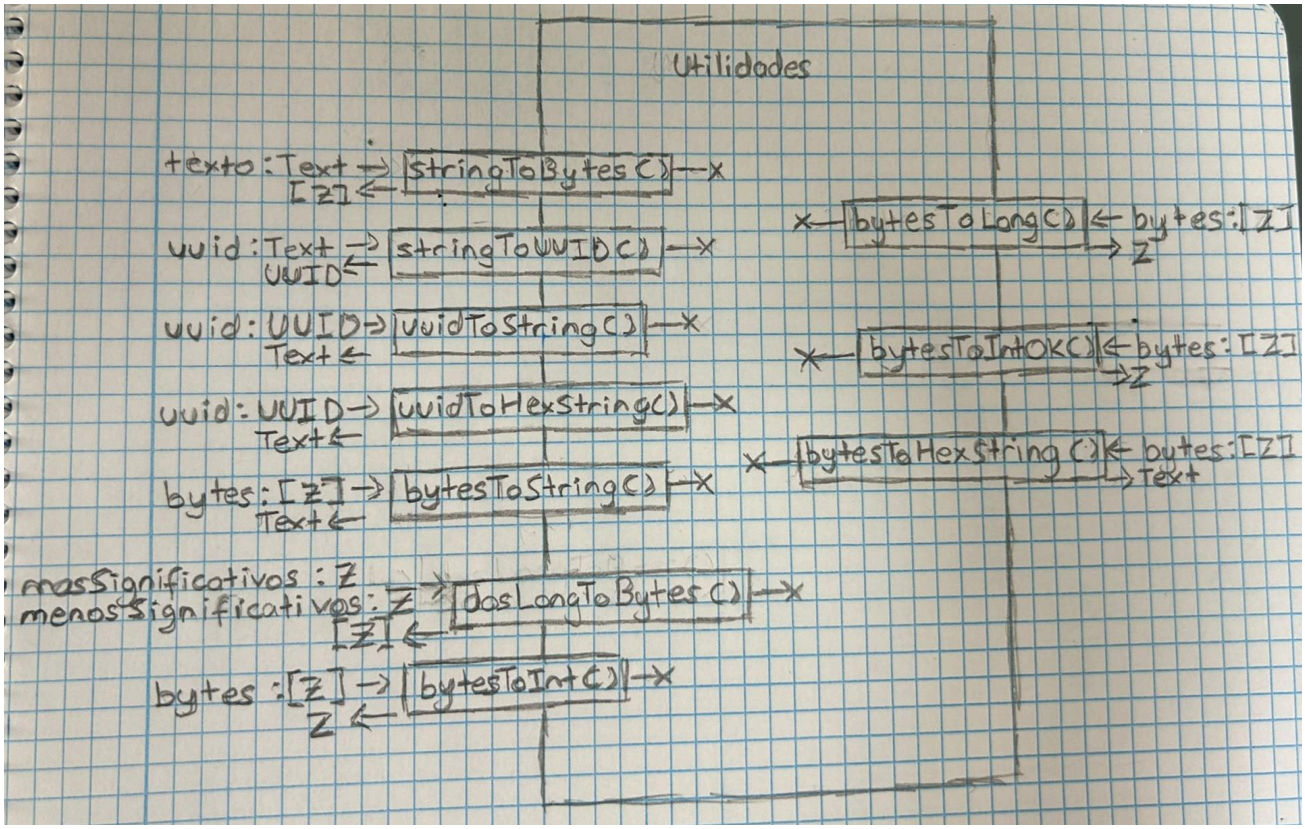
ultimoContadorPorSensor: guarda el último contador de cada sensor para no guardar la misma medición dos veces.

Pantalla: he añadido dos textos (textoUltimaMedicion y textoEstado) para ver en el teléfono la última medición y si se ha guardado en el servidor. Los métodos mostrarMedicion(), mostrarEstado() y mostrarEstadoBusqueda() escriben estos textos.

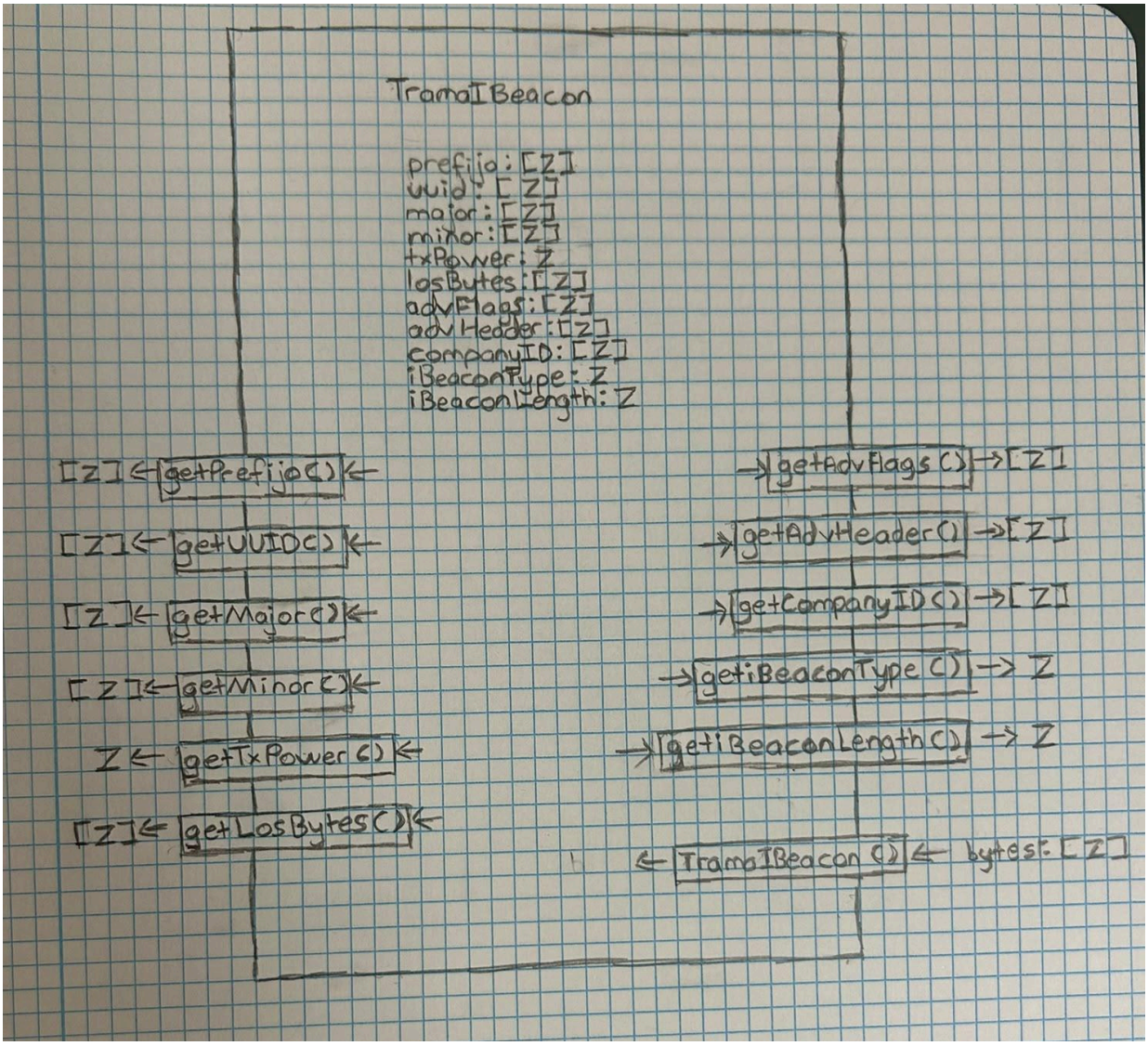
reiniciarEscaneo(): mi teléfono solo avisaba una vez de cada dispositivo. Por eso la aplicación ahora para la búsqueda y la vuelve a empezar cada 10 segundos (INTERVALO_REINICIO_ESCANEO).

tienePermisosBluetooth(): comprueba los permisos. En Android 11 o anterior hace falta el permiso de ubicación.

Utilidades

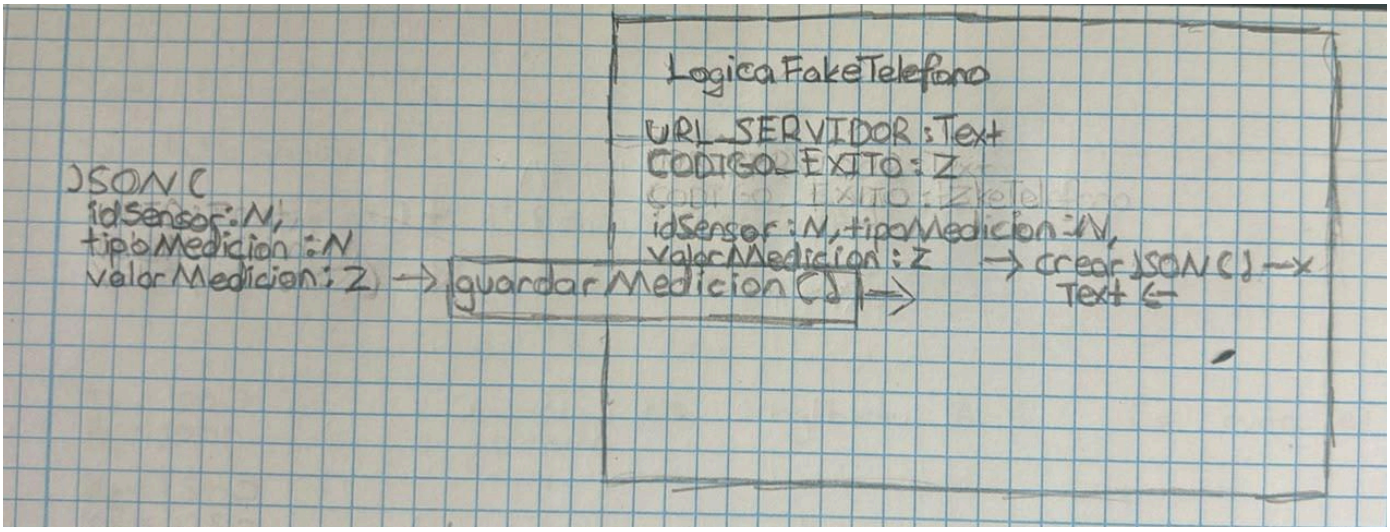


TramaIBeacon



3.3 Lógica del teléfono

LogicaFakeTelefono

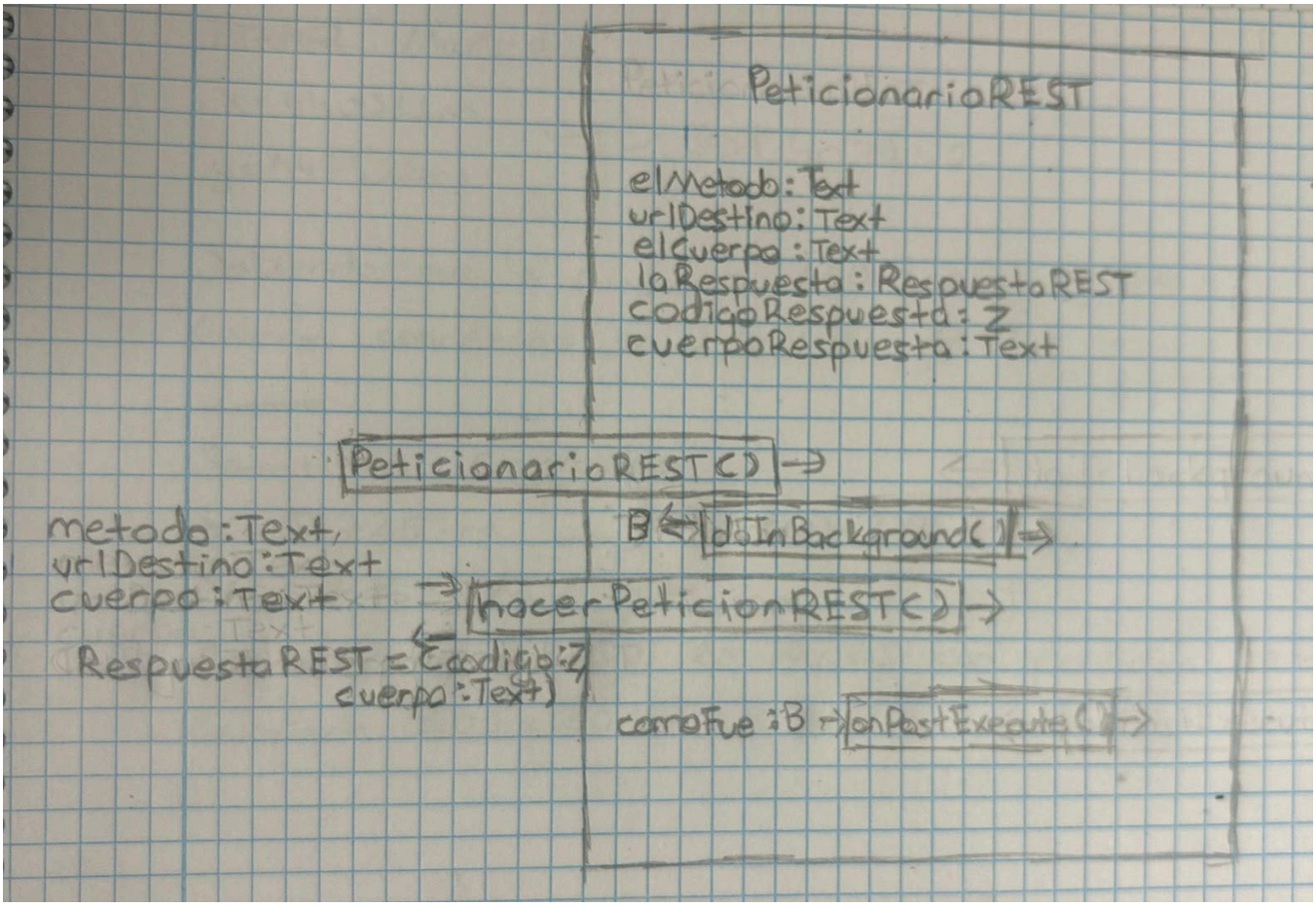


guardarMedicion() ahora recibe idSensor, tipoMedicion y valorMedicion, igual que en la lógica del negocio del servidor. Antes recibía un JSON con el UUID.

He añadido las constantes URL_SERVIDOR (la dirección del servidor) y CODIGO_EXITO (201, el código cuando la medición se guarda bien).

He añadido los métodos privados crearJSON(), esExito() y notificarResultado().

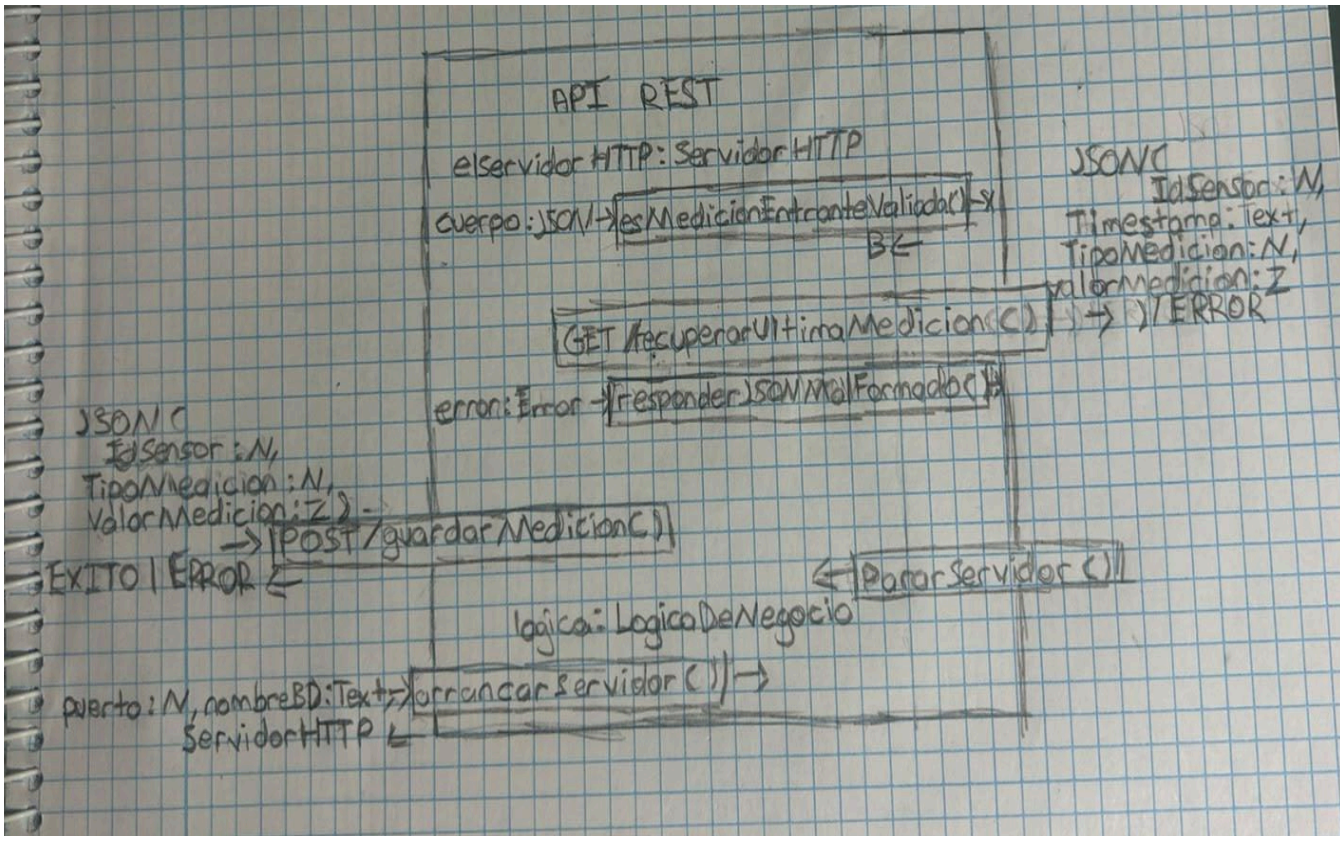
PeticionarioREST



doInBackground() y onPostExecute() son privados, por eso están dentro de la caja.

He añadido la salida de hacerPeticionREST(): RespuestaREST (el código y el cuerpo de la respuesta).

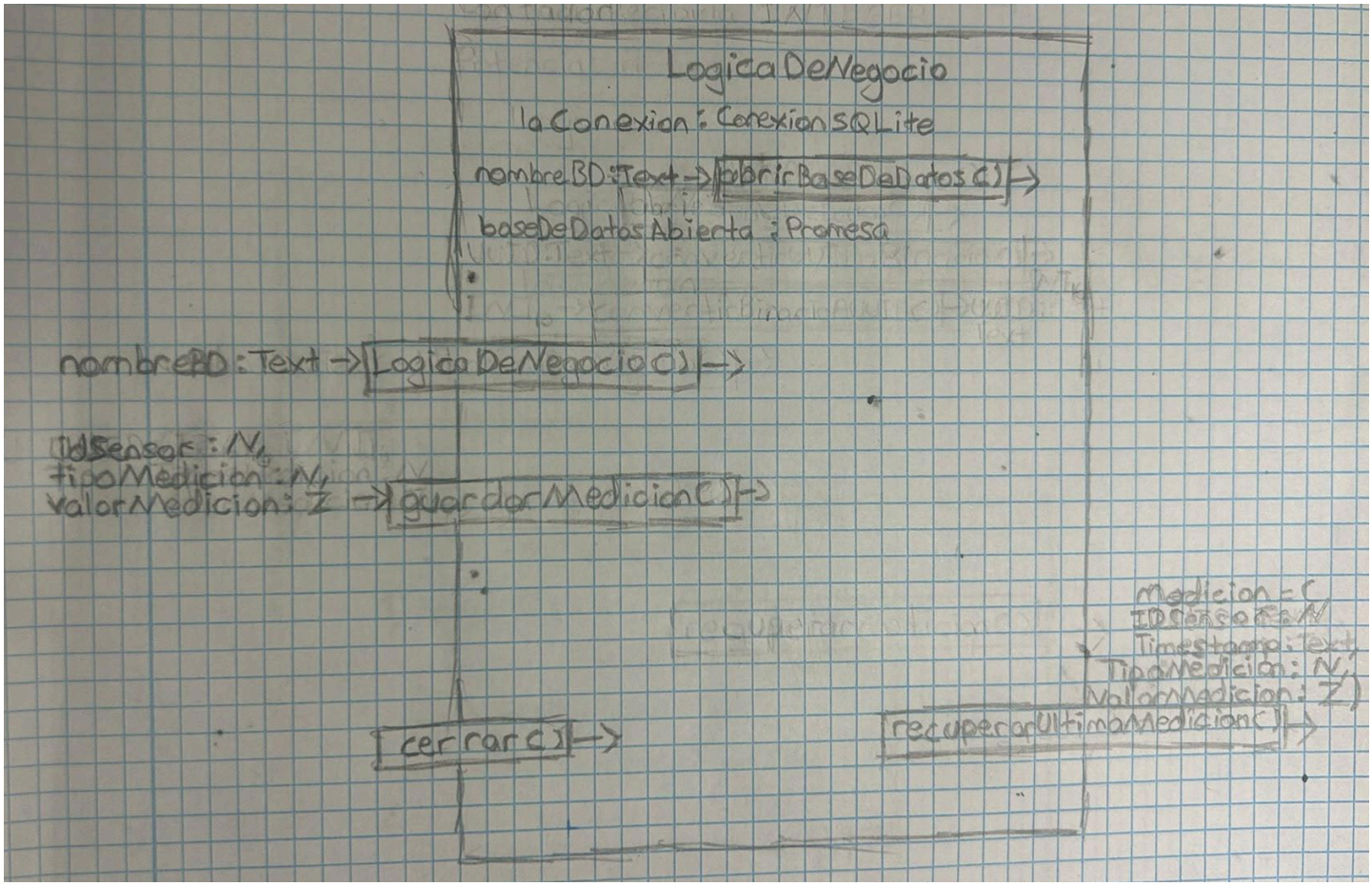
3.4 Servidor REST



Los JSON ahora tienen IdSensor: N en vez de UUID: Text.

He añadido los métodos privados esMedicionEntranteValida() y responderJSONMalFormado(), y los métodos arrancarServidor() y pararServidor().

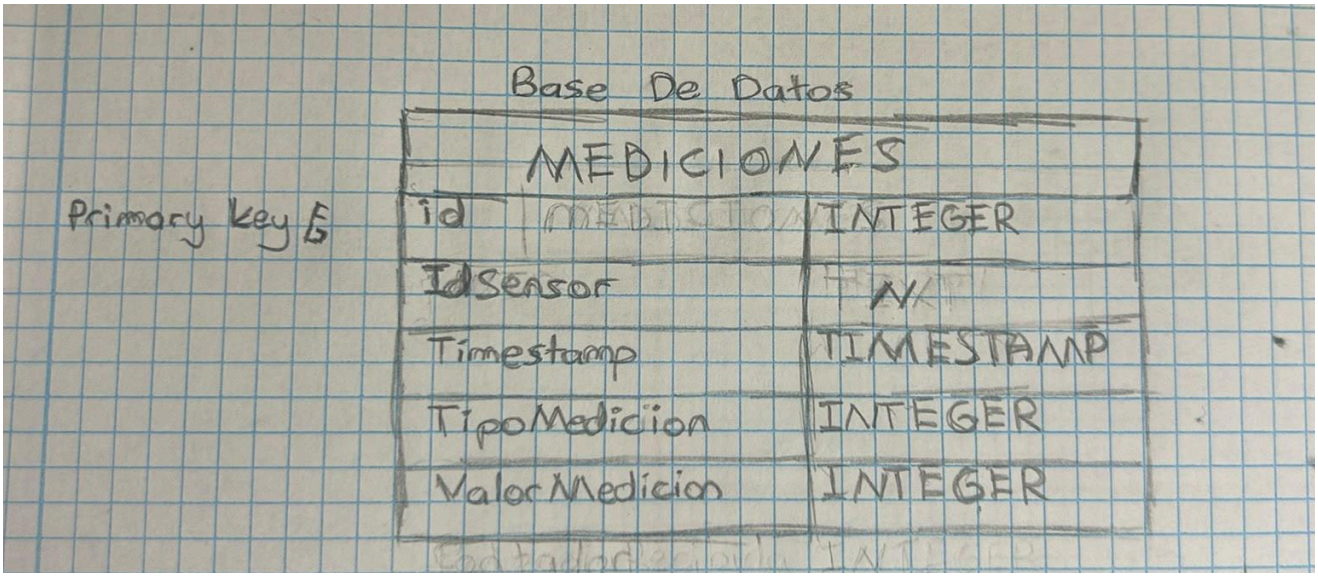
3.5 Lógica del negocio



guardarMedicion() ahora recibe idSensor en vez de UUID.

recuperarUltimaMedicion() devuelve una Medicion, o vacío si todavía no hay mediciones.
He añadido el atributo baseDeDatosAbierta.

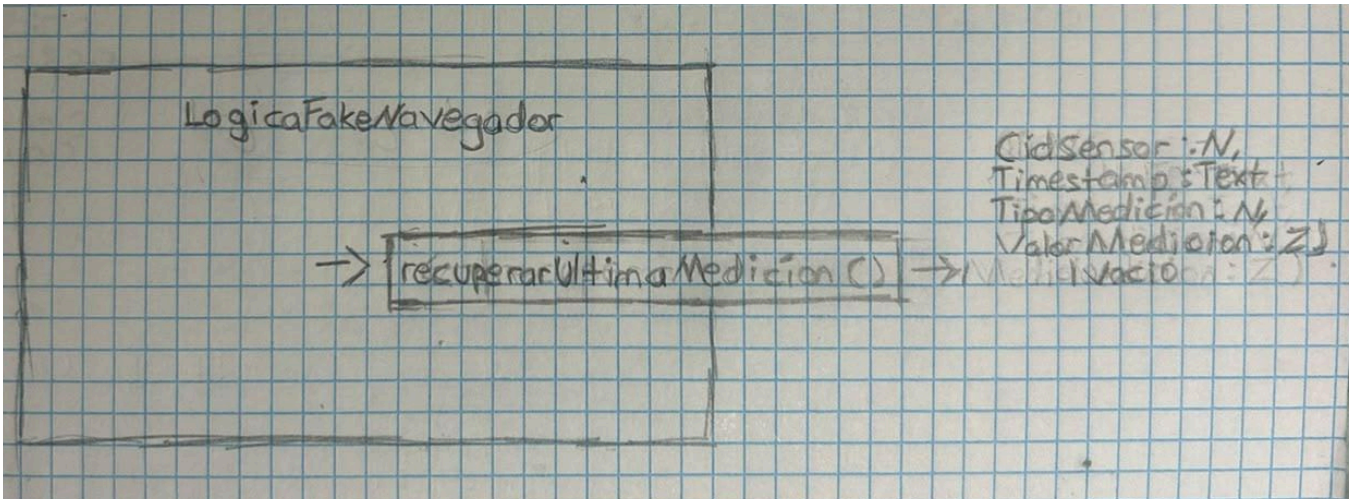
3.6 Base de datos



He quitado la columna UUID y he añadido la columna IdSensor. Así es más fácil saber qué sensor ha hecho cada medición. En el futuro el proyecto puede tener más de un sensor.

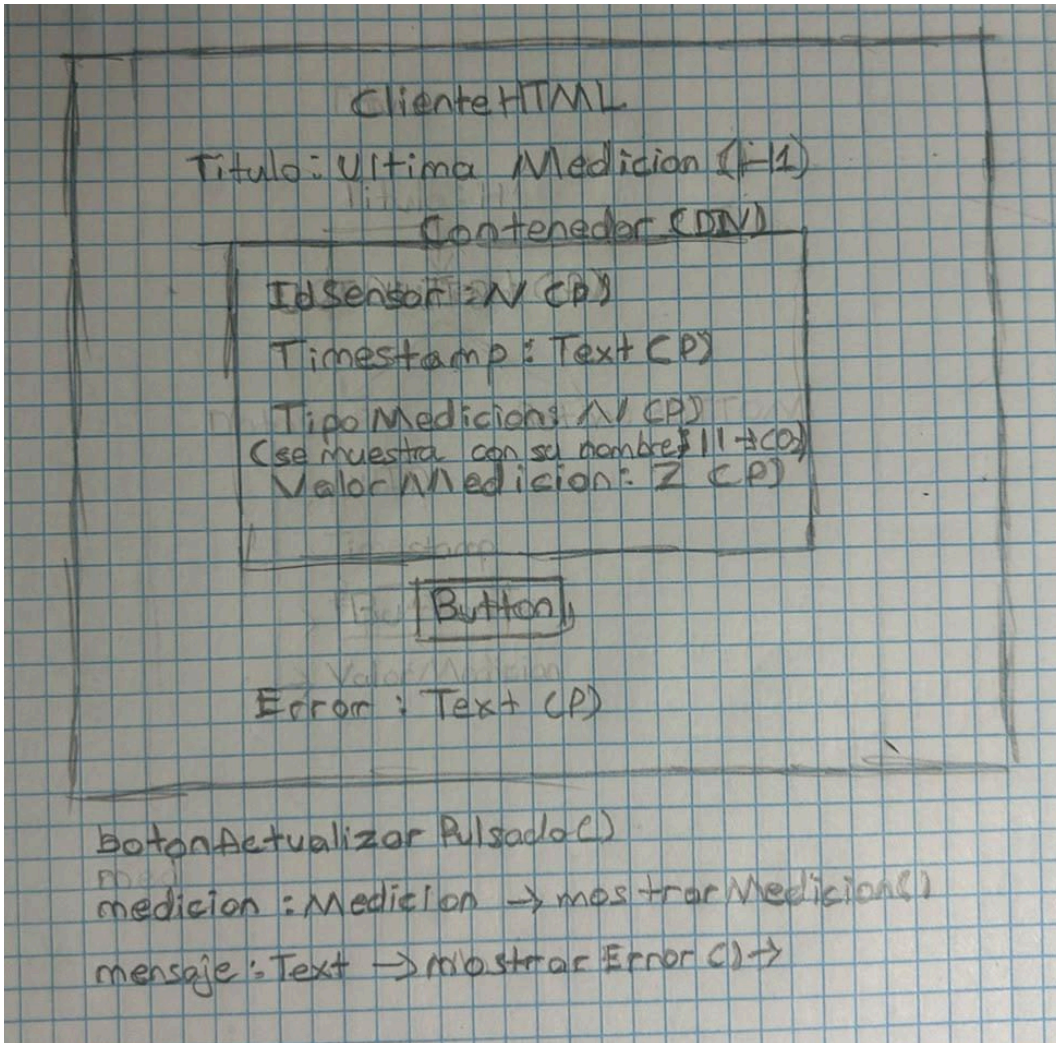
La tabla necesita una clave primaria. Por eso he añadido la columna id. Es más fácil usar un número que aumenta solo (1, 2, 3…) que usar otra columna.

3.7 Lógica del navegador



He cambiado el nombre del método de recuperarMedicion() a recuperarUltimaMedicion(). Ahora es igual que en la lógica del negocio del servidor.

3.8 Web (UX del navegador)



He cambiado UUID por IdSensor, porque ahora la base de datos guarda el número del sensor y no el UUID.
He añadido las funciones de ux.js: botonActualizarPulsado(), mostrarMedicion() y mostrarError()

El tipo de medición se muestra con su nombre (11 → CO2) y la fecha en hora local. Así el usuario entiende los datos más fácilmente. Los datos no cambian, solo cambia cómo se muestran.

4. Diseños que he quitado

He quitado estos diseños porque su código no está en el proyecto del Sprint 0:

- **ServicioEscucharBeacons** y **MainActivity (Servicio):** eran del ejemplo *HolaMundoServicio2021*. En este sprint la búsqueda de la placa se hace desde la MainActivity mientras la aplicación está abierta. El servicio se puede añadir en otros sprints.
- **Web: hacerLogin() y diHola():** eran del ejemplo de PHP. Ahora la web está formada por la lógica del navegador y la UX.